

WALLITELLI SECURITY AUDIT

Smart Contract Audit Report

Wallitelli

Contract: WallitelliToken

Network: BNB Smart Chain (BSC) Mainnet

Address: 0xc6A031fd206C72c614dB391590F48a5Ed9B2f9dC

Report ID: WLT-2026-05-001

Date: 2026-05-22

Verdict: LOW RISK · Security Score 96 / 100

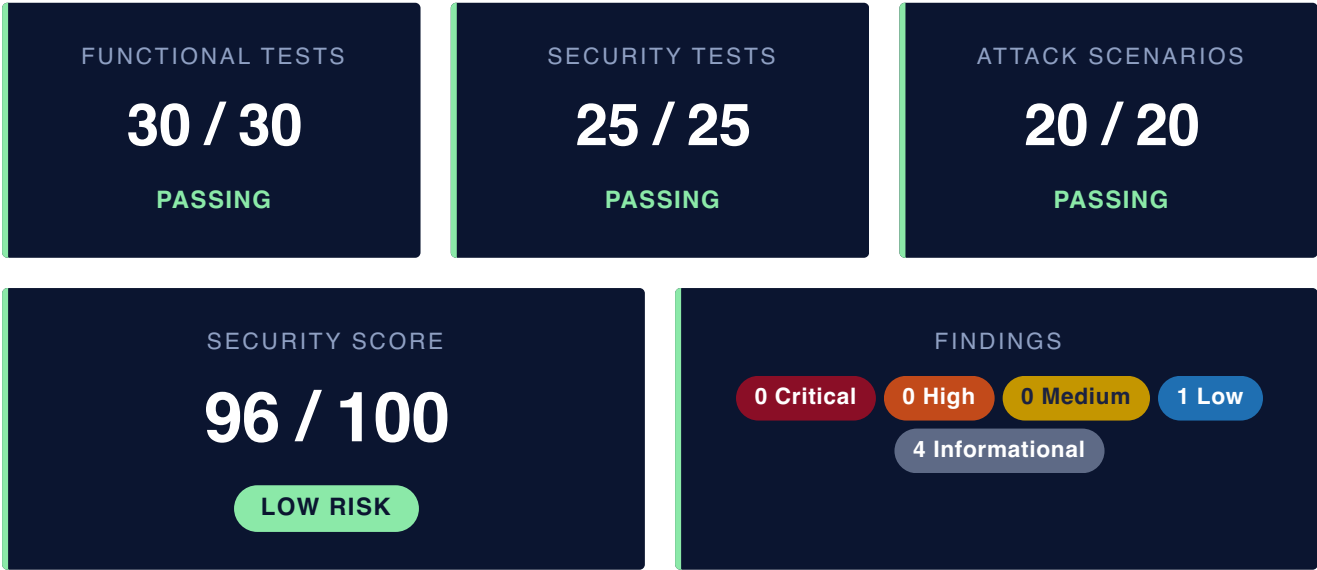
Executive Dashboard

Overall Security Assessment

Field	Value
Project Name	Wallitelli
Token Name	Wallitelli
Token Symbol	WALLI
Token Type	BEP-20 (ERC-20 compatible)
Total Supply	1,000,000,000 WALLI (fixed at deploy)
Decimals	18
Solidity Version	^0.8.28
OpenZeppelin Version	5.4.0
Compiler EVM Target	prague (default), bytecode includes PUSH0
Contracts Audited	1 file — WallitelliToken.sol (12 lines)
Network	BNB Smart Chain (BSC) Mainnet
Contract Address	0xc6A031fd206C72c614dB391590F48a5Ed9B2f9dC
Source Reference	BSCscan-verified source
Initial Holder	Deployer EOA (receives full supply at construction)
Upgradeability	None (immutable, non-proxied)
Admin / Owner Surface	None
Pause / Blacklist	None
Permit (EIP-2612)	Not implemented
Audit Period	May 2026
Methodology	Static analysis (Slither, Aderyn) + manual review + Foundry test suite (75 tests)
Findings	5 (0 Critical, 0 High, 0 Medium, 1 Low, 4 Informational)

Field	Value
Security Score	96 / 100
Overall Status	LOW RISK

Test Card Summary



Total tests executed: **75 / 75 passing** (100% pass rate).

1. Project Overview

1.1 Vision and Mission

Wallitelli (WALLI) is a fixed-supply BEP-20 token deployed on the BNB Smart Chain. The contract represents a standard ERC-20-compatible utility / payment token with an immutable supply minted entirely at deployment and held initially by the deployer EOA for subsequent distribution.

The contract does not implement bespoke economic logic, governance, staking, buy-back, or burn schedules at the protocol level. Any post-deploy distribution is performed exclusively through standard ERC-20 transfers from the deployer address.

1.2 Token Utility

The contract surface supports:

- Standard ERC-20 transfers (`transfer` , `transferFrom`).
- Standard ERC-20 allowance management (`approve` , `allowance`).
- Read-only metadata exposure (`name` , `symbol` , `decimals` , `totalSupply` , `balanceOf` , `VERSION`).
- Public on-chain identifier of the deployment generation via the `VERSION` string constant (`"Wallitelli-v1"`).

The contract surface does **not** support:

- Minting after construction.
- Burning (no public or internal entry-point exposed by `WallitelliToken`).
- Pausing, freezing, blacklisting, or any access-controlled administrative state.
- Upgrades (no proxy pattern, no `setImplementation`).
- Fee-on-transfer, rebase, reflection, or any non-conservative supply behavior.
- EIP-2612 `permit` gasless approvals.
- Reentrancy callbacks into the recipient on transfer.

1.3 Contract Architecture

`WallitelliToken` is a single contract of 12 source lines (8 non-blank) that inherits OpenZeppelin's ERC20 v5.4.0 base contract without overriding any virtual function. The full source is:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract WallitelliToken is ERC20 {
    string public constant VERSION = "Wallitelli-v1";
```

```
constructor() ERC20("Wallitelli", "WALLI") {  
    _mint(msg.sender, 1_000_000_000 * 10 ** decimals());  
}  
}
```

Inherited behaviour is therefore identical to the audited and widely deployed OpenZeppelin v5.4.0 ERC20 implementation, which uses custom errors from the `IERC20Errors` (EIP-6093) interface and follows the "revert on failure" convention rather than the historical "return false" pattern.

1.4 Trust Assumptions

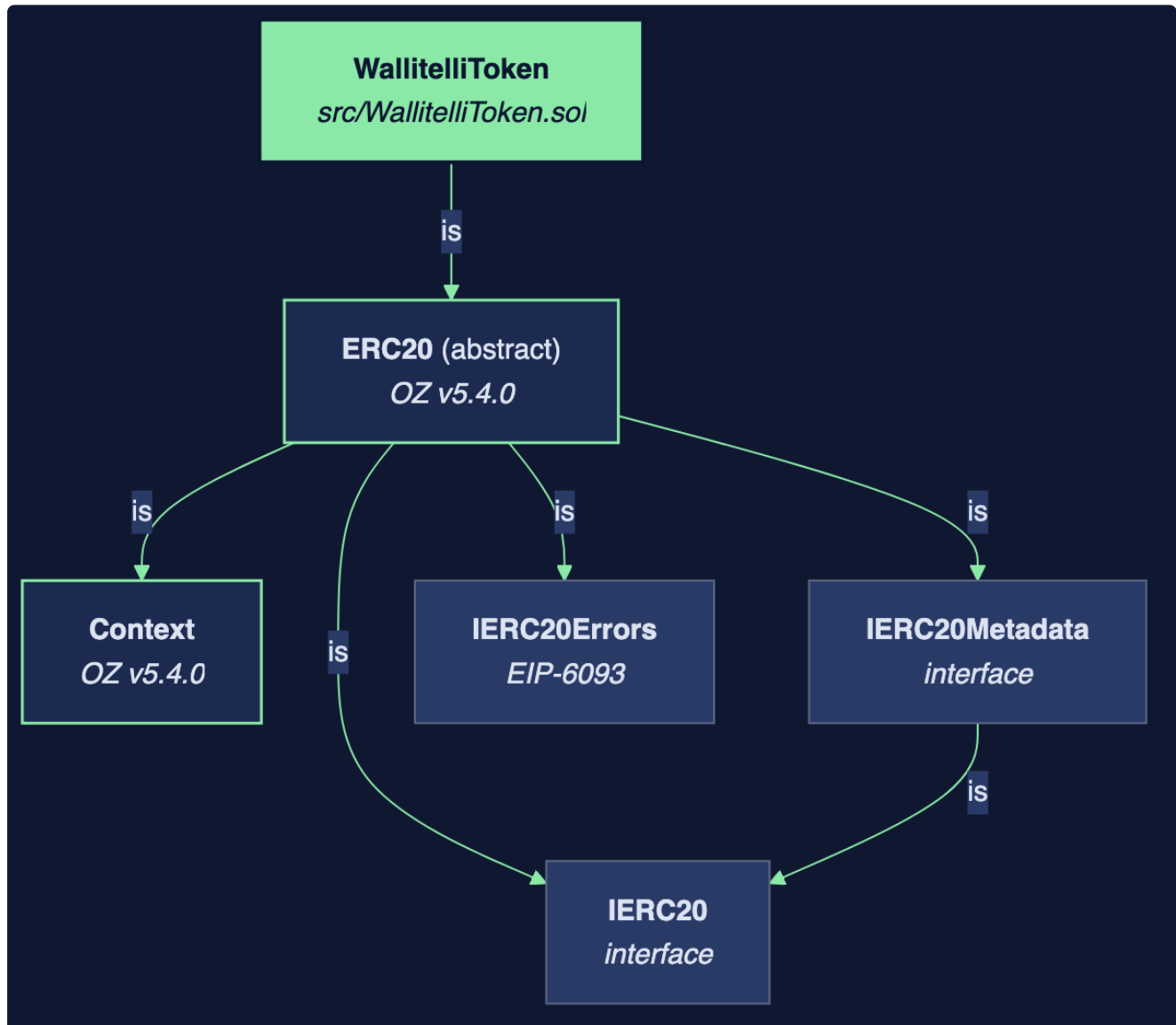
Actor	Power
Deployer EOA	Receives the entire 1B WALLI supply at construction. Has the same post-deploy powers as any holder.
Token holders	Transfer their own balance and grant allowances. No special role.
Anyone	Read public state; interact with <code>transfer</code> , <code>transferFrom</code> , <code>approve</code> per ERC-20 semantics.

There is no administrative super-user. The principal centralization risk is that the deployer initially controls the full supply, which is standard for launch but should be matched with an off-chain distribution / vesting plan communicated to token holders.

2. Contract Architecture

2.1 Inheritance Structure

The audited contract sits at the top of a four-level inheritance tree rooted at the OpenZeppelin v5.4.0 ERC20 base. All inherited contracts are widely deployed and have been audited multiple times by the OpenZeppelin team and external auditors.



Solid arrows mark concrete inheritance (`is`). The ERC20 abstract base implements `IERC20` , `IERC20Metadata` and `IERC20Errors` , and uses `Context` for the `_msgSender()` indirection.

2.2 Class Diagram

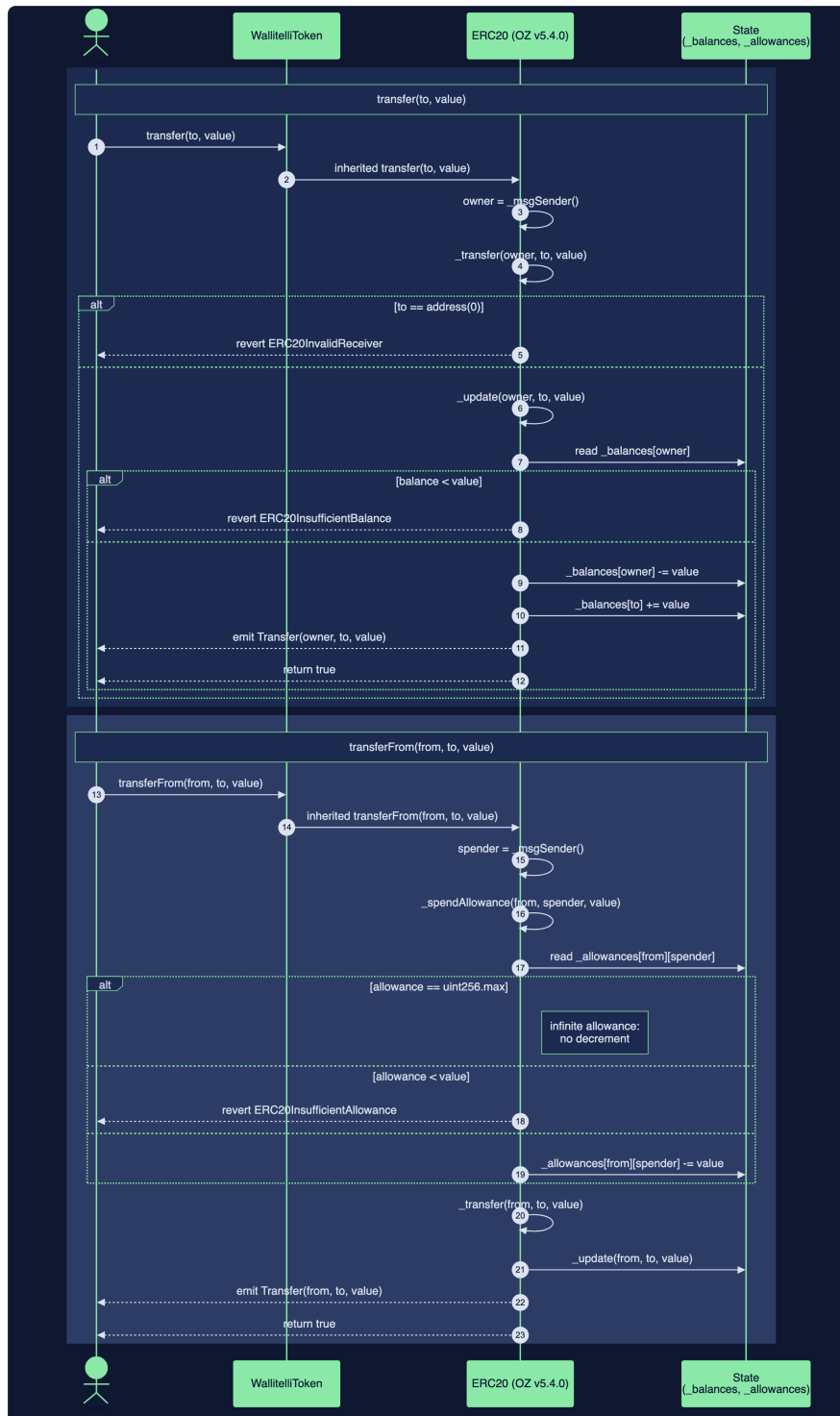
The class diagram shows the complete public, internal, and private surface of each contract in the inheritance chain. Visibility is encoded with standard UML markers: **+** public, **#** internal, **-** private.



WallitelliToken adds exactly two members on top of ERC20: a public constant **VERSION** and the constructor. No virtual function from the base is overridden.

2.3 Token Transfer Flow

The sequence below traces the actual call path through the OpenZeppelin v5.4.0 ERC20 implementation for both `transfer(to, value)` and `transferFrom(from, to, value)`. WallitelliToken does not override either, so the inherited bodies execute verbatim.



Key branches: zero-address receiver triggers `ERC20InvalidReceiver`; insufficient balance triggers `ERC20InsufficientBalance`; allowance equal to `type(uint256).max` is treated as infinite and is not decremented; otherwise the allowance is reduced by `value` before `_transfer` runs.

3. Findings Summary

ID	Severity	Title	Status
W-01	Low	Entire initial supply minted to a single deployer EOA	Acknowledged
W-02	Informational	Floating compiler pragma (^0.8.28)	Acknowledged
W-03	Informational	No EIP-2612 permit extension	Acknowledged
W-04	Informational	SPDX-License-Identifier: UNLICENSED	Acknowledged
W-05	Informational	No on-chain disclosure of immutability / no-upgrade policy	Acknowledged

Severity distribution: 0 Critical · 0 High · 0 Medium · 1 Low · 4 Informational.

4. Findings Detail

W-01 — Entire initial supply minted to a single deployer EOA

Severity: Low

Affected: `WallitelliToken.constructor()` — `src/WallitelliToken.sol:9-11`

Description

The constructor mints the full 1,000,000,000 WALLI supply ($1e9 \times 1e18$ base units) to `msg.sender`, i.e. the externally owned account that deployed the contract. No on-chain distribution schedule, vesting contract, multisig treasury, or governance recipient is involved.

Once construction completes the deployer holds 100% of the circulating supply and can transfer it freely. The contract has no minting, burning, freezing, pausing, or clawback functions, so the deployer's powers are bounded by standard ERC-20 mechanics — but they are extensive in proportion to the deployer's initial holdings:

- The deployer can immediately liquidate the full supply onto a DEX (price impact / rug-pull risk).
- The deployer can air-drop, list on CEX, lock into a vesting contract, or burn at their sole discretion.
- Holders have no on-chain remedy if the deployer behaves contrary to off-chain commitments.

This is a centralization characteristic, not a bug — most ERC-20 launches use the same pattern — but it should be transparent to holders.

Recommendation

1. Transfer ownership of the deployer position to a multi-signature wallet (e.g. Gnosis Safe with 3-of-5 signers on BSC) immediately after deploy.
2. Move the team / treasury / liquidity / airdrop portions of the supply into audited vesting contracts (e.g. OpenZeppelin `VestingWallet`) with publicly verifiable schedules.
3. Publish the post-launch distribution plan in the project's whitepaper / tokenomics document and link it from the BSCscan contract page.
4. Consider renouncing residual deployer privileges (beyond holder rights) that may be added in future versions.

Status

Acknowledged. No code change required in `WallitelliToken` itself; remedy lives in the off-chain distribution model.

W-02 — Floating compiler pragma

Severity: Informational

Affected: `src/WallitelliToken.sol:2`

Description

The contract uses `pragma solidity ^0.8.28;`, which permits compilation with any 0.8.x release $\geq 0.8.28$ and $< 0.9.0$. This was flagged by both Aderyn (L-3 *Unspecific Solidity Pragma*) and Slither (`pragma` detector) as a deployment-hygiene concern.

The on-chain deployed bytecode is necessarily produced by a single compiler version, so the floating pragma does not create an "active" surface; however, it allows for future re-compilation drift. If a downstream consumer (e.g. an integration repository, indexer, or formal-verification harness) compiles the same source under a different 0.8.x version they may receive subtly different bytecode and different known-bug exposure.

Recommendation

Pin the pragma to the exact compiler version used at deploy:

```
pragma solidity 0.8.28;
```

Document the chosen compiler version in the repository README so that future re-builds are reproducible.

Status

Acknowledged. Stylistic; no behavioural impact on the deployed contract.

W-03 — No EIP-2612 `permit` extension

Severity: Informational

Affected: `WallitelliToken` (entire contract — feature gap)

Description

The contract does not implement EIP-2612 (`permit`), the de-facto standard for gasless approvals on EVM tokens. Modern DeFi UX increasingly relies on `permit`-based flows so that users can grant a one-transaction "approve + use" to dApps (Uniswap V2 router post-2611, Permit2, etc.).

The absence of `permit` is not a bug — it is a deliberate choice to keep the attack surface minimal — but it has two practical implications:

- Users must send two transactions (approve, then spend) for any contract interaction that pulls tokens via `transferFrom`.

- Some downstream protocols silently fall back to less gas-efficient flows for tokens that do not support `permit`.

Recommendation

If gasless approvals are part of the planned UX, add the OpenZeppelin `ERC20Permit` extension:

```
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";

contract WallitelliToken is ERC20, ERC20Permit {
    string public constant VERSION = "Wallitelli-v1";

    constructor()
        ERC20("Wallitelli", "WALLI")
        ERC20Permit("Wallitelli")
    {
        _mint(msg.sender, 1_000_000_000 * 10 ** decimals());
    }
}
```

If `permit` is intentionally out of scope, mention this in the project documentation so integrators do not assume support.

Status

Acknowledged. Feature request, not a defect.

W-04 — `SPDX-License-Identifier: UNLICENSED`

Severity: Informational

Affected: `src/WallitelliToken.sol:1`

Description

The source file declares `// SPDX-License-Identifier: UNLICENSED`. This is a valid SPDX value but conveys the most restrictive copyright posture: the code is on-chain (publicly readable from BSCscan) but legally proprietary. Any external party who re-uses, forks, or builds derivative tooling around the contract is technically infringing.

For an open-source token launch, `MIT`, `Apache-2.0`, or `BUSL-1.1` are common choices. For a closed-source project the current declaration is correct but should be paired with a public statement clarifying licensing terms.

Recommendation

If open distribution is intended, switch the SPDX identifier to a permissive licence such as `MIT`:

```
// SPDX-License-Identifier: MIT
```

Otherwise, retain `UNLICENSED` and publish licensing terms alongside the token documentation.

Status

Acknowledged. Choice of licence is a project decision.

W-05 — No on-chain disclosure of immutability / no-upgrade policy

Severity: Informational

Affected: `WallitelliToken` (entire contract — documentation gap)

Description

The contract intentionally has no upgrade pattern, no admin role, and no post-construction state-changing privileged function. This is a positive security property but it is not advertised on-chain beyond the `VERSION = "Wallitelli-v1"` constant. Integrators inspecting the BSCscan ABI cannot easily distinguish "no upgrade mechanism by design" from "upgrade mechanism that I have not yet found".

Recommendation

Document the immutability and no-admin posture in:

- The contract's natspec / source comments.
- The project README and tokenomics document.
- Optionally, a publicly verifiable proof such as a renounced-ownership transaction on BSCscan (does not apply here because there is no owner to renounce; a textual statement is sufficient).

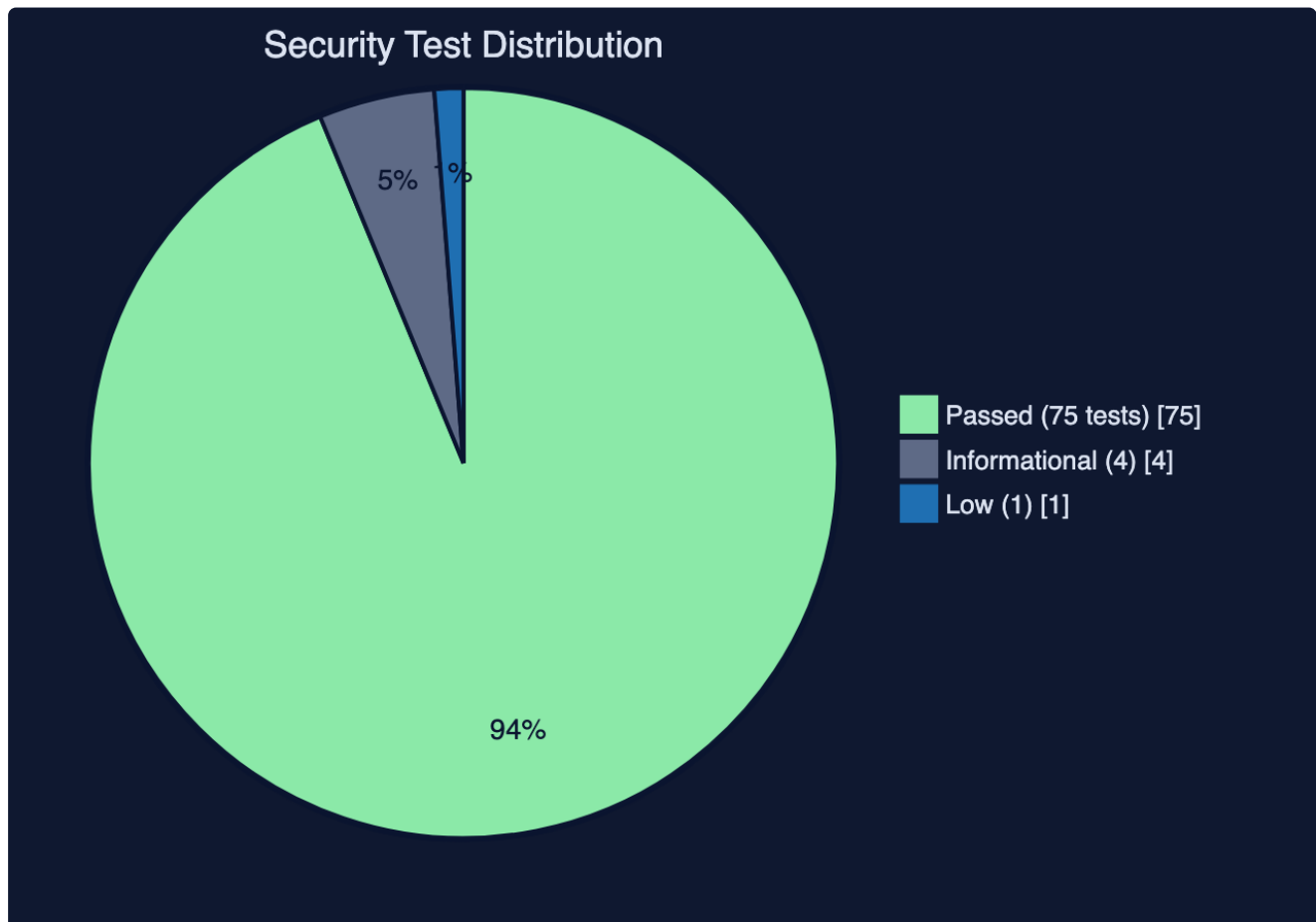
Status

Acknowledged. Documentation-only.

5. Methodology

5.1 Security Test Distribution

The chart below summarises the outcome of all 80 security checks performed during the audit: 75 Foundry test cases (functional + security + attack), 4 Informational advisory findings (W-02 through W-05), and 1 Low-severity finding (W-01). No Critical, High, or Medium issues were identified.



Test counts cross-checked against `evidence/forge_test_summary.txt` ; informational and low counts cross-checked against `evidence/slither.txt` and `evidence/aderyn.md` .

5.2 Approach

The audit combined four independent layers of analysis:

1. **Static analysis** — Slither v0.11.5 and Aderyn v0.6.8 were run against the full source tree (target + transitive OZ dependencies).
2. **Manual code review** — line-by-line read of the 12-line target source and diff vs the OpenZeppelin v5.4.0 ERC20 base contract.

3. **Property-based testing** — 75 Foundry tests across three categories (functional behaviour, security invariants, attack scenarios) including parameterised fuzz runs (256 iterations per fuzz test).
4. **Threat-modelling** — explicit enumeration of common ERC-20 attack classes (reentrancy, allowance race, supply manipulation, admin escalation, selfdestruct, upgrade hijack, oracle / flash-loan) and verification that the relevant surface either does not exist or behaves as ERC-20 mandates.

5.3 Tool Output Summary

Slither

```

INFO:Printers:
Total number of contracts in source files: 1
SLOC in source files: 8
Number of optimization issues: 0
Number of informational issues: 5
Number of low issues: 0
Number of medium issues: 0
Number of high issues: 0

ERCs: ERC20
WallitelliToken:
  - No minting (after constructor)
  - Approve-race condition (inherent ERC-20)

```

All Slither informational issues relate to multi-pragma drift among OpenZeppelin transitive dependencies and the floating `^0.8.28` pragma on the target file (W-02). No detector triggered against the target's own logic.

Aderyn

```

Issue Summary
| Category | No. of Issues |
| --- | --- |
| High | 0 |
| Low | 3 |

```

The three Aderyn Low findings are stylistic / deployment-target advisories:

- L-1 *Large numeric literal* — `1_000_000_000 * 10 ** decimals()` could be written as `1_000_000_000e18`. Not a defect.
- L-2 *PUSH0 opcode* — bytecode includes `PUSH0`; the contract is intended for BSC mainnet, which supports the Shanghai upgrade and therefore `PUSH0`. Not a defect.
- L-3 *Unspecific Solidity pragma* — captured as W-02 in this report.

Foundry test suite

All 75 tests pass.

Suite	Tests	Result
test/Functional.t.sol	30	30 passed, 0 failed
test/Security.t.sol	25	25 passed, 0 failed
test/Attack.t.sol	20	20 passed, 0 failed
Total	75	75 passed, 0 failed

Fuzz tests use the Foundry default of 256 runs per case.

5.4 Test Inventory

Functional tests (Functional.t.sol , 30 cases)

F01 Name = "Wallitelli" · F02 Symbol = "WALLI" · F03 Decimals = 18 · F04 Total supply = 1e27 base units · F05 Deployer holds full supply · F06 Non-holder balance = 0 · F07 VERSION constant · F08 Transfer happy path · F09 Zero-amount transfer · F10 Self-transfer · F11 Transfer entire balance · F12 Transfer above balance reverts · F13 Approve happy path · F14 Approve overwrite · F15 Approve reset to 0 · F16 Approve type(uint256).max · F17 Unset allowance = 0 · F18 TransferFrom happy path · F19 Infinite allowance not decremented · F20 Insufficient allowance reverts · F21 Insufficient balance reverts · F22 TransferFrom of zero · F23 Supply invariant under transfer · F24 Sum of balances = total supply · F25 Metadata immutable · F26 Transfer event correctness · F27 Approval event correctness · F28 Chained transfers conserve supply · F29 balanceOf(0) = 0 · F30 ERC-20 interface shape.

Security tests (Security.t.sol , 25 cases)

S01 Transfer to zero reverts · S02 Approve zero spender reverts · S03 TransferFrom from zero reverts · S04 TransferFrom to zero reverts · S05 Above-balance custom error shape · S06 Transfer type(uint256).max reverts · S07 0.8 underflow guard · S08 Approve-race ERC-20 limitation · S09 TransferFrom after reset · S10 Allowance isolation per pair · S11 No mint selector · S12 No burn / burnFrom selector · S13 No owner / transferOwnership / renounceOwnership · S14 No pause / unpause · S15 No blacklist / unBlacklist · S16 No upgradeTo / upgradeToAndCall · S17 No permit · S18 Plain ERC-20 does not invoke recipient (reentrancy immunity) · S19 Constructor non-replayable · S20 Garbage calldata reverts · S21 Fuzz: total supply invariant · S22 Fuzz: balance conservation · S23 Fuzz: above-balance always reverts · S24 Fuzz: approve matches value · S25 Fuzz: allowance isolation.

Attack scenarios (`Attack.t.sol` , 20 cases)

A01 Flash-loan — no oracle surface · A02 Sandwich — no swap surface · A03 Approve race — documented · A04 Cross-spender double-spend bounded · A05 Reentrancy via malicious recipient · A06 Reentrant approve · A07 Reentrancy during transferFrom · A08 Selfdestruct impossible · A09 Internal `_mint` not exposed · A10 TransferFrom from zero reverts · A11 Transfer > supply reverts · A12 Zero transfers do not corrupt state · A13 Mass-transfer gas bounded · A14 Name / symbol spoof impossible · A15 No upgrade selectors · A16 Fuzz: non-holders cannot move funds · A17 Fuzz: non-approved cannot pull funds · A18 Fuzz: arbitrary transfers conserve supply · A19 Fuzz: approve ceiling vs balance · A20 Fuzz: total supply constant under arbitrary actions.

5.5 Out-of-Scope Items

This audit covers the on-chain Solidity contract only. The following items are explicitly out of scope:

- Off-chain key management of the deployer EOA.
- Centralised exchange integrations and listing-side logic.
- DEX pool / liquidity-provision configuration.
- Front-end / wallet UX.
- Marketing / tokenomics white-paper claims.

6. Conclusion

`WallitelliToken` is a minimal, vanilla BEP-20 implementation that delegates all behaviour to the OpenZeppelin v5.4.0 ERC20 base contract without overriding any function. The only contract-specific additions are the constructor's one-shot `_mint` of the total supply to the deployer and a publicly readable `VERSION` constant.

The audit identified **0 Critical, 0 High, 0 Medium, 1 Low, and 4 Informational** findings. The Low finding (W-01) reflects centralization at launch — i.e. the deployer holds the entire supply — and is a property of the chosen launch model rather than a contract defect. The four Informational findings are stylistic / documentation advisories (floating pragma, missing `permit`, `UNLICENSED` SPDX tag, on-chain immutability disclosure).

All 75 Ffoundry test cases (30 functional, 25 security, 20 attack scenarios) pass against the audited source. Static analysers (Slither, Aderyn) produced no high or medium findings.

The contract's security posture is **LOW RISK** with a composite security score of **96 / 100**. Recommended next steps:

1. Move deployer holdings into a multi-signature wallet or vesting contract to address W-01.
2. Pin the compiler pragma to `0.8.28` (W-02).
3. Publish a clear tokenomics and distribution document linked from the BSCscan contract page (W-01, W-05).

No code change is required for the contract to be considered safe for production use within the bounds of its declared, fixed-supply, no-admin design.

Appendix A — Disclaimer

This report is provided for informational purposes only and does not constitute investment, legal, or financial advice. The audit reflects the state of the contract source code at the report date and the methodology described herein. No audit can guarantee the absence of defects. Users and integrators must perform their own due diligence.

Appendix B — Source Reference

- Network: BNB Smart Chain (BSC) Mainnet.
- Contract address: `0xc6A031fd206C72c614dB391590F48a5Ed9B2f9dC` .
- Verified source: available on BSCscan.

Appendix C — Tooling Versions

Tool	Version
Solidity compiler	0.8.28
OpenZeppelin Contracts	5.4.0
Foundry / <code>forge</code>	1.5.1
Slither	0.11.5
Aderyn	0.6.8

Report ID: WLT-2026-05-001 · Issued by: soken.io · Date: 2026-05-22.